


# NPTEL Course on "Computer System Optimization for ML"

Lecture #36

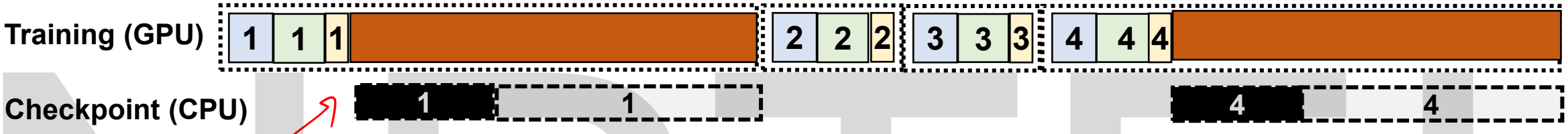
NPTEL

# Agenda

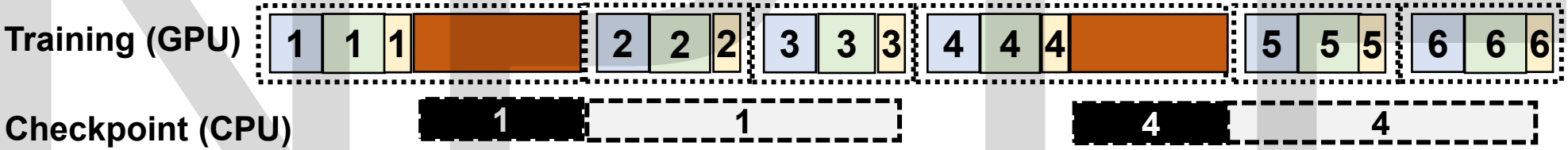
- **Fault Tolerance in Machine Learning (contd.)**

NPTEL

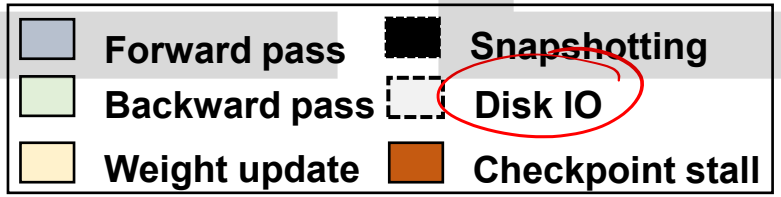
# Pipelining to Reduce Overhead

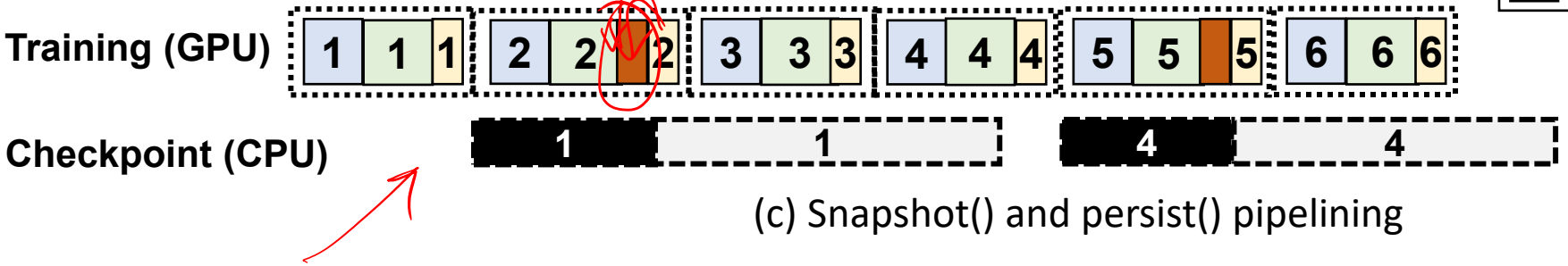



(a) Baseline : Synchronous checkpointing




(b) Only persist() pipelining



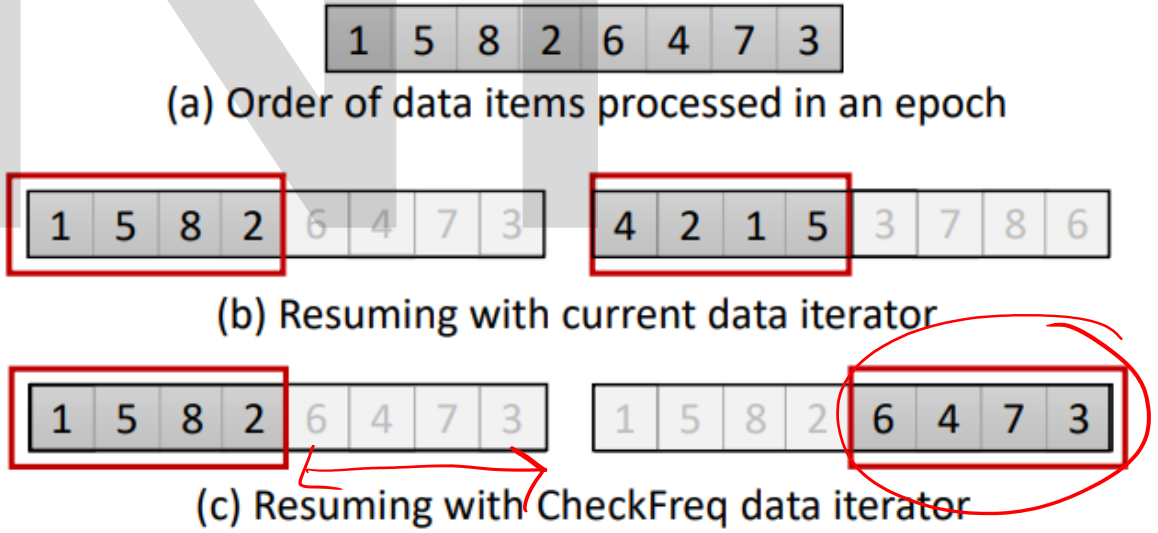



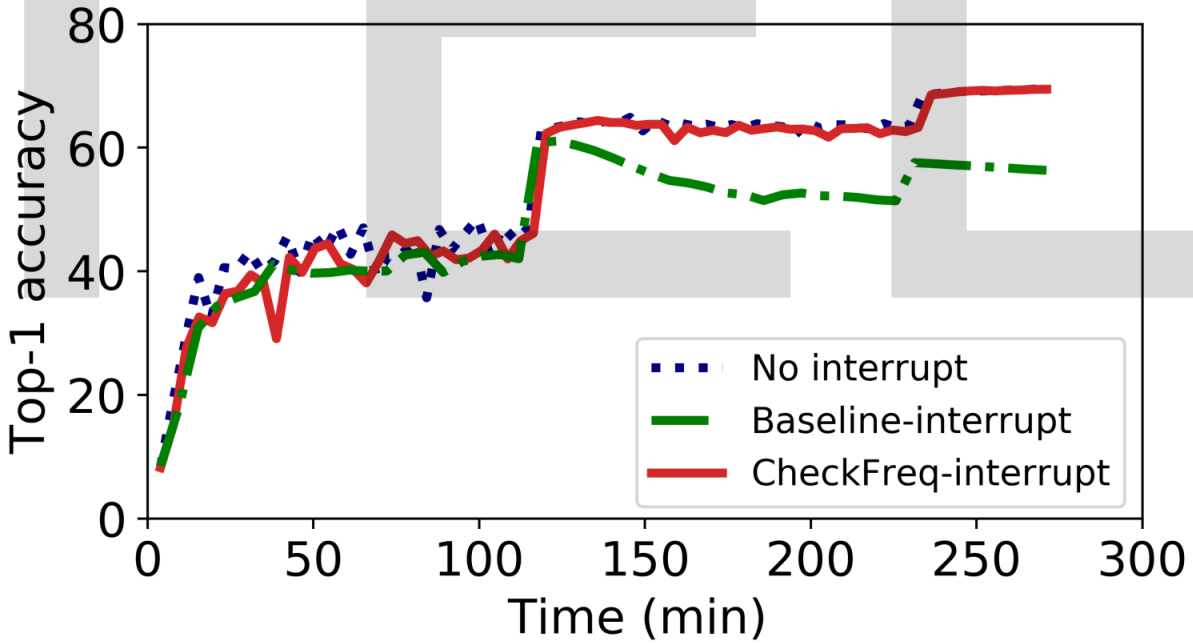

(c) Snapshot() and persist() pipelining

NDTF

# Resumable Data Iterator

- **Feeds data item to the GPU process**
- **Has maintain the invariant that each epoch sees a data item exactly once, in a shuffled order**



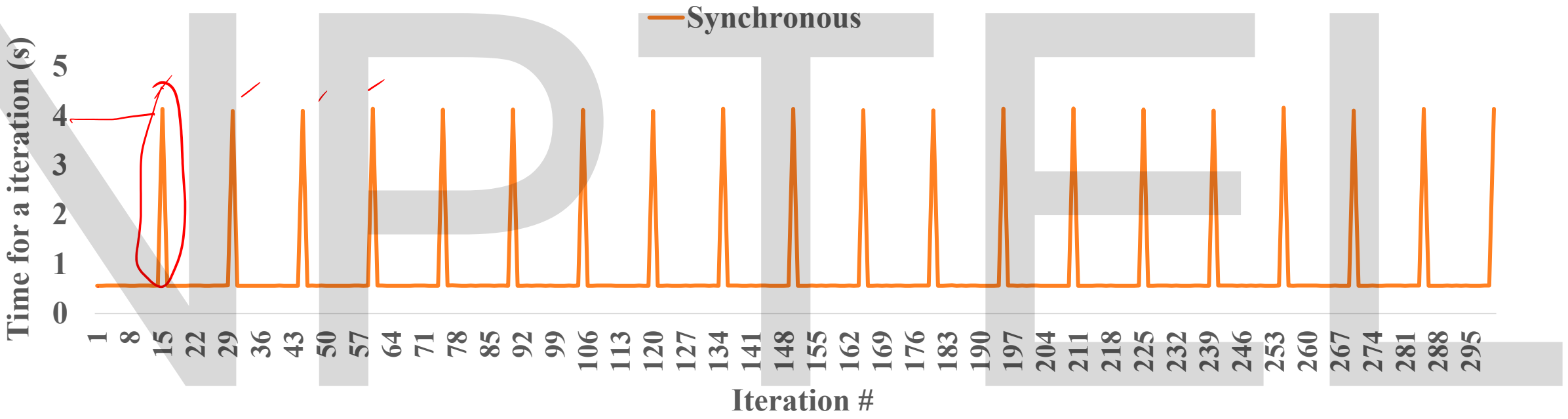
# Adaptive Rate Tuning

- Job environment can change, interferences can be introduced
- Run a background process for frequency estimation and modify checkpointing frequency as needed

| *Setting* | Isolated | Static | Adaptive |
|---|---|---|---|
| *Overhead* | 5% | 35% | 5% |
| *Frequency (# iterations)* | 14 | 14 | 19 |

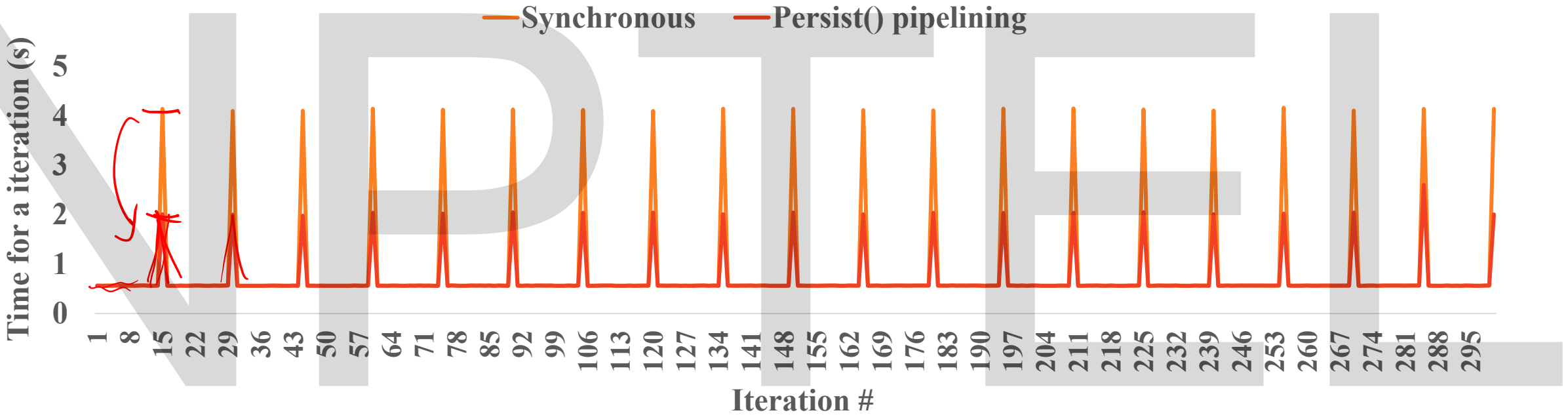
NPTEL

# CheckFreq Reduces Checkpoint Stalls



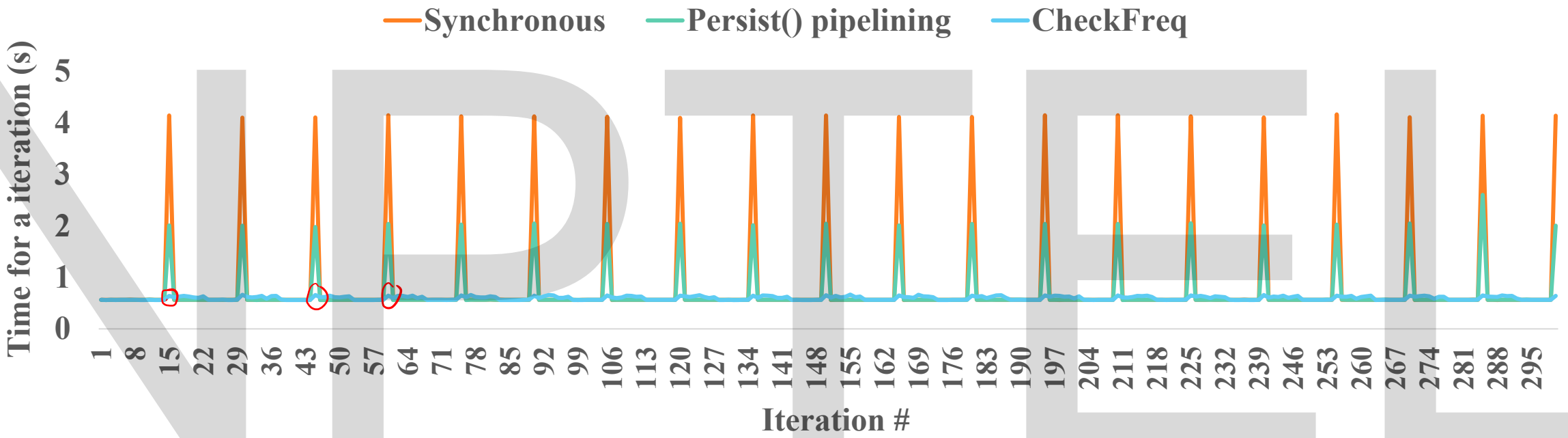

INTEL

# CheckFreq Reduces Checkpoint Stalls




**Performing asynchronous IO reduces checkpoint cost by 2x but still results in significant stalls**

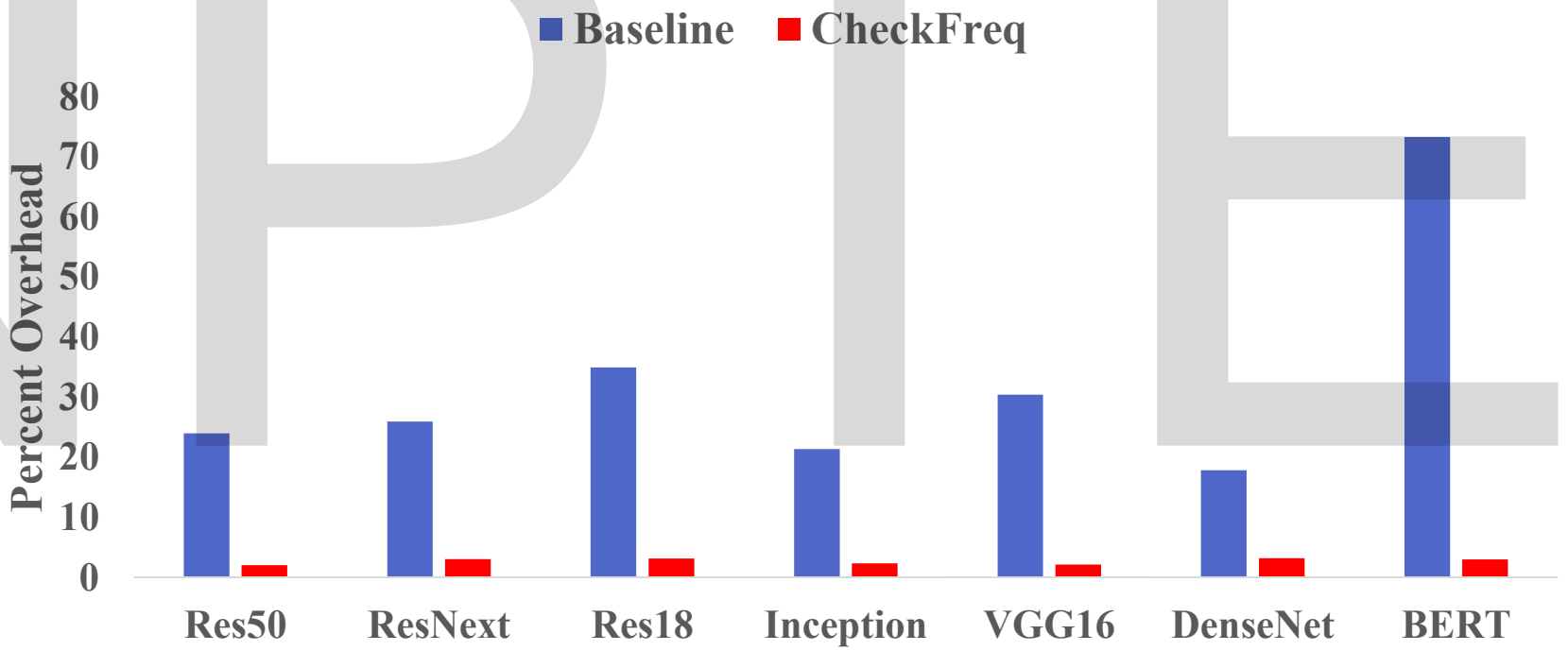
# CheckFreq Reduces Checkpoint Stalls




**CheckFreq further reduces stalls by carefully pipelining checkpointing with compute**

# Overall Training Overhead

- When the baseline checkpointing mechanism is performed at a frequency chosen by CheckFreq, it introduces 20 – 70% overhead in training time

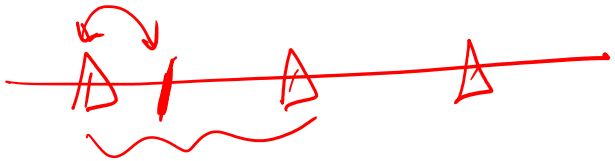



NPTEL

# CheckFreq Lowers Recovery Time

- **CheckFreq reduces recovery time during an interruption from hours to seconds**

| Model | Epoch-based (s) | CheckFreq (s) |
|---|---|---|
| Res18 | 840 | 5 |
| Res50 | 2100 | 24 |
| VGG16 | 5700 | 25 |
| ResNext | 7080 | 32 |
| DenseNet | 2340 | 7 |
| Inception | 3000 | 27 |
| BERT | 4920 | 85 |



# Check-N-Run (NSDI'22)

- **High write bandwidth needed for large model checkpoints**
- **Large storage capacity needed to store checkpoints**

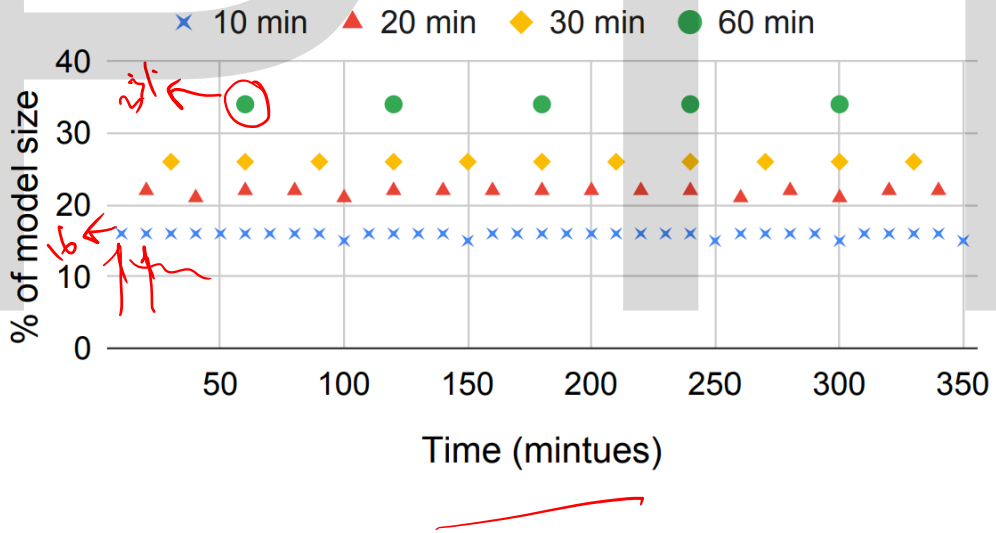
NPTEL

Differential checkpointing

Quantized checkpoints

# Reducing Write BW with Differential Checkpointing

- **Motivation: model accesses are sparse for Recommender systems**
- **Lots of sparse features – every iteration modifies only a small portion of model parameters**




NPTEL

# Differential Checkpoint

- Start with a single checkpoint taken as a full baseline checkpoint, including all the embedding vectors
- System tracks all modified vectors to create a differential view of the embedding vectors
- Each differential checkpoint would then store only the vectors that were modified since the baseline checkpoint

NPTEL

# Approaches for Differential Checkpointing

- **One-Shot Differential Checkpoint**

NPTEL

Drawback?

**Checkpoint size increases as time progresses**

# Approaches for Differential Checkpointing

- One-Shot Differential Checkpoint
- Consecutive Incremental Checkpoint
  - Store changes since last checkpoint

Drawback?

**Need to manage all previous checkpoints and recursively apply updates**
**Move all data from remote store to local host – b/w cost**

NPTEL

# Approaches for Differential Checkpointing

- One-Shot Differential Checkpoint
- Consecutive Incremental Checkpoint
  - Store changes since last checkpoint
- **Intermittent Differential Checkpoint**
  - **Take a full snapshot of model at regular interval so that subsequent differentials are small enough**
  - **At the end of each checkpoint interval, it estimates the expected cumulative size of future checkpoints if another differential checkpoint is taken, compared with the total expected size if a full checkpoint is taken**

NPTEL




# THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)